MATHLIB5: An Immutable System of Safety Boundaries for Verified Symbolic Compute

Ahmad Ali Parr
SnapKitty Collective · SNAPKITTYWEST · ORCID 0009-0006-1916-5245
Technical Paper · Sealed 2026-07-05 · License CC-BY-4.0

Abstract

MATHLIB5 is a verified symbolic-compute substrate that evaluates summations and matrix contractions written in a small APL fragment and proves, at every stage of a ten-layer pipeline, that the emitted code matches its closed-form mathematics. Each layer is a safety boundary that stamps a signed receipt; receipts are appended to a write-once (WORM) chain whose head is anchored to the Bitcoin ledger through an OP_RETURN transaction. This paper states the architecture, the verification chain, and the cryptographic seals — a SHA-256 fingerprint, an Ed25519 signature (the Plasma Gate), and the MATH5:<sha256> anchor — by which any reader can independently confirm the authenticity of this document and its accompanying software.

1. Architecture Overview

The Verified Symbolic Compute Pipeline (VSCP) transforms an APL expression through ten sequentially certified layers: (1) APL frontend, (2) typed intermediate representation, (3) Liquid Haskell refinement types, (4) Lean 4 closed-form proofs, (5) C-- emission, (6) static analysis, (7) first-order policy/FOL, (8) a P/NP swarm for open proofs, (9) the WORM ledger, and (10) a completeness metric. The output contract of each layer is the input contract of the next, so a single broken boundary is caught before any artifact can propagate.

1. APL frontend	Numeric expression entered in a small APL fragment.
2. IR	Typed, shape-aware intermediate representation.
3. Liquid Haskell	Refinement types prove values stay in promised bounds.
4. Lean 4	Closed-form mathematical identity proven as a theorem.
5. C-- emission	Verified expression lowered to C-- code.
6. Static analysis	Checks bounds and initialization.
7. FOL / policy	Correctness claim rendered as first-order logic.
8. P/NP swarm	Open proof gaps posted as solvable problems.
9. WORM ledger	Receipts appended to a write-once chain.

10. Completeness	Honest score of how much is proven.

2. The Verified Kernel and Bridge

A small Coq-like verified kernel stores polynomials as tagged values and reduces them by the rules of arithmetic. A foreign-function bridge lets this kernel call audited C and Lean code across one confined, tag-checked boundary, bounding the trusted surface of the heterogeneous stack.

3. Cryptographic Seals

Authenticity rests on three independently checkable artifacts. Their values for this release are fixed below and are reproduced by `verify_anchors.py` in the repository.

3.1 SHA-256 fingerprint

The fingerprint covers the LaTeX source tree of the full monograph and the verification receipt:

```
88313c7cea5c462eec80069f12fc28d771465c3aebeec2a79f4661d502a03491
```

3.2 Ed25519 public key (Plasma Gate)

```
18d816694de0deae621e913177bdfa3547e5d4cc2f9d91dfdcc3a16d03d02141
```

3.3 Bitcoin OP_RETURN anchor

A Bitcoin transaction embeds the payload below (70 bytes, within the 80-byte standard limit). The 6-byte `MATH5:` prefix namespaces MATHLIB5 anchors:

```
MATH5:88313c7cea5c462eec80069f12fc28d771465c3aebeec2a79f4661d502a03491
```

4. Independent Verification

A verifier performs the following, with no trust placed in the authors:

- Fetch the Bitcoin transaction carrying the MATH5: OP_RETURN and recover the expected hash.
- Recompute SHA-256 over the local document; it must equal the recovered hash.
- Verify the Ed25519 signature over that hash under the published public key.
- Confirm the OP_RETURN prefix is MATH5: and the hash matches.

All four must hold for the document to be accepted as the anchored release. Tampering with any intermediate artifact forces either an Ed25519 forgery (assumed hard) or a rewrite of the Bitcoin anchor (economically bounded by proof-of-work).

5. Reproducibility

The software is bit-for-bit reproducible from a pinned commit. The verification gate aborts on any unverified artifact, so a published artifact is always accompanied by a complete certificate chain or does not exist at all. Unproven lemmas are recorded as open debt in `sorrydb` and exposed to the P/NP swarm, keeping the completeness score honest.

6. Scope and Relation to Other Systems

MATHLIB5 is domain-specific: it certifies summation and matrix contraction for a small APL fragment, and does not aim to replace general proof libraries or production C compilers. Its distinguishing combination is an end-to-end verified pipeline together with public, on-chain provenance — a property not provided by CompCert, CertiCoq, Lean mathlib, or Isabelle.

7. Metadata and Citation

This record is accompanied by `.zenodo.json` describing authors, license (CC-BY-4.0), keywords, and the related software repository. To cite: A. A. Parr, “MATHLIB5: An Immutable System of Safety Boundaries for Verified Symbolic Compute,” SnapKitty Collective, 2026, sealed under SHA-256 88313c7cea5c462e….

Repository: https://github.com/SNAPKITTYWEST/mathlib5 · ORCID: 0009-0006-1916-5245 · Sealed 2026-07-05